

Reviewer Pack — Minimal Rank of Algebraic-Geometric Codes vs SOS Degree for Ma...

Entry 1063142987f7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 14:04:14 UTC

Minimal Rank of Algebraic-Geometric Codes vs SOS Degree for Max-CUT

Entry ID: 1063142987f7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 14:04:14 UTC

1. Conjecture

Field A (mathematical branch): Algebraic-Geometric Coding Theory **Field B** (complexity object): Sum-of-squares Lower Bounds for Max-CUT

Statement:

[‘The minimal rank of an algebraic-geometric code constructed from a MAX-CUT instance is polynomially related to the SOS degree required for an approximation ratio better than 0.878.’, ‘For any given MAX-CUT instance, there exists a corresponding algebraic-geometric code such that the minimal rank of this code is at most some polynomial function of the instance size n and a constant factor related to the SOS degree needed for an approximation ratio better than 0.878.’]

Rationale (proposer’s reasoning):

[‘Algebraic-geometric codes have been known to possess strong structural properties, which might provide a new lens into understanding the complexity of MAX-CUT.’, ‘If such a relationship holds, it would suggest that problems related to SOS degrees in max-CUT can be approached using algebraic geometry, potentially leading to new algorithms or lower bounds.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6117dd1411317dd1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The relationship between the minimal rank of an algebraic-geometric code and the SOS degree for a MAX-CUT instance is supported if, for all seeds, the ratio of the minimal rank to the SOS degree is less than or equal to 1.2 and the mean difference is less than or equal to 3 across at least 100 instances.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebro-geometric codes AND Sum-of-squares lower bounds - MAX-CUT AND SOS degree approximation ratio better than 0.878 AND algebro-geometric coding theory - minimal rank of codes constructed from MAX-CUT instances AND related to SOS degree

Top relevant hits considered: - [<http://arxiv.org/abs/1603.04558v2>] An algebro-geometric study of special values of hypergeometric functions ${}_3F_2$ - [<http://arxiv.org/abs/quant-ph/9804043v2>] Dense Quantum Coding and a Lower Bound for 1-way Quantum Automata - [<http://arxiv.org/abs/2011.02044v5>] Circuit lower bounds for low-energy states of quantum code Hamiltonians - [<http://arxiv.org/abs/2404.18142v1>] Revisiting Majumdar-Ghosh spin chain model and Max-cut problem using variational quantum algorithms - [<http://arxiv.org/abs/2206.09204v1>] Approximating Max-Cut on Bounded Degree Graphs: Tighter Analysis of the FKL Algorithm - [<http://arxiv.org/abs/2402.12631v2>] Theoretical Approximation Ratios for Warm-Started QAOA on 3-Regular Max-Cut Instances at Depth $p = 1$ - [<http://arxiv.org/abs/2401.11160v7>] Quasi-Perfect and Distance-Optimal Codes Sum-Rank Codes - [<http://arxiv.org/abs/1202.0085v3>] Affine cartesian codes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        factor = A[i][i]
        for j in range(i, n):
            A[i][j] /= factor
        for k in range(i+1, n):
```

```

        factor = A[k][i]
        for j in range(i, n):
            A[k][j] -= factor * A[i][j]
    return A

def rank(matrix):
    A = [row[:] for row in matrix]
    r = gaussian_elimination(A)
    rank = 0
    for row in r:
        if any(row[j] != 0 for j in range(len(row))):
            rank += 1
    return rank

def sos_degree(n):
    # Placeholder function to compute SOS degree
    # This is a dummy implementation and should be replaced with actual computation
    return n * (n + 1) // 2

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    incidence_matrix = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    np = sum(sum(row) for row in incidence_matrix)
    if np % 2 != 0:
        return {
            "metric_name": "Ratio of Minimal Rank to SOS Degree",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Invalid MAX-CUT instance"
        }

    minimal_rank = rank(incidence_matrix)
    sos_deg = sos_degree(n)

    return {
        "metric_name": "Ratio of Minimal Rank to SOS Degree",
        "metric_value": Fraction(minimal_rank, sos_deg),
        "instances_tested": 1,
        "conjecture_holds": None,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or list(range(2, 30*3 + 1))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(result["metric_value"] is not None and result["conjecture_holds"] is None for result in results):

```

```

mean_ratio = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["metric_value"] <= Fraction(3, 2))
print(f"RESULT: SUPPORTED mean={mean_ratio} std=0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["metric_value"] > Fraction(3, 2))
    print(f"RESULT: FALSIFIED counterexample='Ratio too high' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TRIAL: {'metric_name': 'Ratio of Minimal Rank to SOS Degree', 'metric_value': None, 'instances_tested': 1, 'MAX-CUT instance'}

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5d0f6201.py", line 84, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5d0f6201.py", line 67, in run_trial
    minimal_rank = rank(incidence_matrix)
                   ^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5d0f6201.py", line 40, in rank
    r = gaussian_elimination(A)
       ^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5d0f6201.py", line 31, in gaussian_elimination
    A[i][j] /= factor

```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it was unable to complete the required computations and verify the conjecture. | next: Review the test code for potential causes of the crash and ensure that it can handle edge cases properly. Once fixed, re-run the test with a valid MAX-CUT instance.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15469
2	propose	ollama_remote	glm4:latest	0	0	14443

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	propose	ollama_remote	glm4:latest	0	0	13287
4	preregistration	ollama_remote	glm4:latest	0	0	9396
5	novelty	ollama_remote	glm4:latest	0	0	8448
6	novelty	ollama_remote	glm4:latest	0	0	9633
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21845
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9912
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10087
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8859
11	judge	ollama_remote	glm4:latest	0	0	13740

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 135118 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/1063142987f7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1063142987f7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1063142987f7.tar.gz` (if generated)